

PO Long Text New Persistency

4-Step Implementation Guide

SE11 Table → SE24 Handler Class → SE19 BAdI → Test & Verify

SAP ERP / S4HANA | Single System | STXH/STXL → ZPO_LONGTEXT | 2026-06-28

■ System-Specific Finding: TDNAME for EKPO (item text) on this system is constructed WITHOUT a space between PO number and item number. Correct for this system: CONCATENATE iv_ebeln iv_ebelp INTO lv_tlname. (e.g. '450007774400010') Standard documentation says: EBELN + SPACE + EBELP — NOT valid here. Header EKKO TDNAME = EBELN only (e.g. '4500077744') — unchanged.

The 4 Steps at a Glance

Step	Transaction	Object Name	Purpose
1	SE11	ZPO_LONGTEXT	Create custom table — stores plain readable text lines
2	SE24	ZCL_PO_LONGTEXT_HANDLER	Create handler class — reads STXL via READ_TEXT, writes to ZPO_LONGTEXT
3	SE19	ZCL_PO_TEXT_BADI_IMPL	Create BAdI — fires on PO save, calls handler automatically
4	ME21N + SE16N	—	Test end-to-end — create PO with text, verify ZPO_LONGTEXT populated

Text ID (TDID) Reference — Confirmed for This System

TDOBJECT	TDID	Description	TDNAME Format	EBELP in ZPO_LONGTEXT
EKKO	F01	PO Header General Note	EBELN only (e.g. 4500077744)	00000
EKKO	F02	PO Header Delivery Text	EBELN only	00000
EKPO	F01*	PO Item General Note	EBELN+EBELP no space (450007774400010)	item no. e.g. 00010
EKPO	F09*	PO Item GR Text	EBELN+EBELP no space	item no.

* Confirm TDID from the STXH row you found. The TDID shown in STXH result is the one to use in READ_TEXT calls.

SE11 — Create ZPO_LONGTEXT Table

Why This Table Is Needed

SAP stores all long texts in STXH (header) and STXL (content). The content in STXL is COMPRESSED — it cannot be read by plain SQL SELECT. Every read requires calling function module READ_TEXT which decompresses on the fly. ZPO_LONGTEXT solves this by storing the same text in plain, human-readable format in a flat table — one row per text line.

Table Definition — SE11 → Database Table → ZPO_LONGTEXT → Create

Field	Key	Type	Length	Description
MANDT	✓	CLNT	3	Client
EBELN	✓	CHAR	10	Purchase Order Number
EBELP	✓	NUMC	5	PO Item — 00000 for Header texts
TDOBJECT	✓	CHAR	10	Text Object: EKKO (header) or EKPO (item)
TDID	✓	CHAR	4	Text ID: F01, F02, F09 etc.
TDSPRAS	✓	LANG	1	Language Key: D for German
LINE_COUNTER	✓	NUMC	5	Line sequence: 00001, 00002, 00003...
TDFORMAT		CHAR	2	Format indicator — * = normal text line
TDLINE		CHAR	132	Text line content — PLAIN readable text
MIGRATED_AT		DATS	8	Date this row was written
MIGRATED_BY		CHAR	12	User who wrote this row
SOURCE		CHAR	1	M = Migration program, D = Dual-write BAdI

Technical Settings

Setting	Value	Reason
Delivery Class	A	Application data — written by customer processes
Data Browser	Display/Maintenance Allowed	Allows SE16N browsing
Enhancement Category	Can Be Enhanced (Deep)	Future extensibility
Buffering	Not buffered	Table is frequently written — no buffer

What the Table Looks Like After Population

ZPO_LONGTEXT – sample data after BAdI dual-write:

EBELN	EBELP	TDOBJECT	TDID	TDSPRAS	LINE	COUNTER	TDLINE	SOURCE
-------	-------	----------	------	---------	------	---------	--------	--------

4500077744	00000	EKKO	F01	D	00001	PO header note line 1	D
4500077744	00000	EKKO	F01	D	00002	PO header note line 2	D
4500077744	00010	EKPO	F01	D	00001	PO item 10 general note - test line 1	D
4500077744	00010	EKPO	F01	D	00002	PO item 10 general note - test line 2 te	D

Key points:

EBELP = 00000 for EKKO (header texts)

EBELP = 00010 for EKPO item 10 texts

TDLINE is PLAIN TEXT – no compression, no binary characters

LINE_COUNTER drives ORDER BY for correct text sequence

SOURCE = D means written by BAdI (Dual-write)

Recommended Secondary Index

SE11 → ZPO_LONGTEXT → Indexes → Create Index: ZPO_LT_I01 Fields: EBELN, TDOBJECT, TDID Why: Most queries filter by EBELN + TDOBJECT + TDID. Without this index, large table scans slow down BDC programs.

■ Step 1 complete when ZPO_LONGTEXT is Active in SE11 with all fields and primary key correct.

What This Class Does

This is the engine of the whole solution. It takes a PO number, text object, text ID and language — calls SAP function module READ_TEXT to decompress the text from STXL — then writes each plain text line as a separate row into ZPO_LONGTEXT. It is called by both the migration program and the BAdI.

Class Definition — SE24 → ZCL_PO_LONGTEXT_HANDLER → Create

Method	Visibility	Purpose
WRITE_TO_PERSISTENCE	Public / Static	Reads text from STXL via READ_TEXT. Deletes old rows. Inserts new plain rows into ZPO_LONGTEXT.
READ_FROM_PERSISTENCE	Public / Static	SELECT * FROM ZPO_LONGTEXT for a given EBELN. Returns plain text lines. No READ_TEXT needed.

WRITE_TO_PERSISTENCE — Full ABAP Code

■ System-specific: TDNAME for EKPO uses NO SPACE between EBELN and EBELP on this system. EKKO: lv_tlname = iv_ebeln (e.g. '4500077744') EKPO: CONCATENATE iv_ebeln iv_ebelp INTO lv_tlname (e.g. '450007774400010')

```

CLASS zcl_po_longtext_handler DEFINITION PUBLIC FINAL CREATE PUBLIC.

    PUBLIC SECTION.

        CLASS-METHODS write_to_persistence

            IMPORTING iv_ebeln      TYPE ebeln

                     iv_ebelp      TYPE ebelp      DEFAULT '000000'

                     iv_tldobject  TYPE tldobject

                     iv_tldid      TYPE tldid

                     iv_tdspras    TYPE tdspras.

        CLASS-METHODS read_from_persistence

            IMPORTING iv_ebeln      TYPE ebeln

            RETURNING VALUE(rt_lines) TYPE TABLE OF zpo_longtext.

ENDCLASS.

CLASS zcl_po_longtext_handler IMPLEMENTATION.

    METHOD write_to_persistence.

```

```

DATA: lt_lines      TYPE TABLE OF tline,

      lv_tdtype     TYPE tdtype,

      lv_counter    TYPE numc5,

      ls_persist    TYPE zpo_longtext.

" Build TDNAME – system-specific: NO space for EKPO

IF iv_tdtype = 'EKPO'.

    lv_tdtype = iv_ebeld.

ELSE.

    " EKPO: concatenate WITHOUT space (confirmed for this system)

    CONCATENATE iv_ebeld iv_ebelp INTO lv_tdtype.

ENDIF.

" Read and decompress text from STXH/STXL via READ_TEXT

CALL FUNCTION 'READ_TEXT'

EXPORTING

    id          = iv_tdid

    language    = iv_tdsprg

    name        = lv_tdtype

    object      = iv_tdtype

TABLES

    lines       = lt_lines

EXCEPTIONS

    OTHERS      = 4.

IF sy-subrc <> 0 OR lt_lines IS INITIAL.

    RETURN. " No text found – nothing to write

ENDIF.

" Full refresh: delete existing rows for this text

DELETE FROM zpo_longtext

```

```

WHERE ebeln      = iv_ebeln

      AND ebelp      = iv_ebelp

      AND tdoobject = iv_tdoobject

      AND tddid      = iv_tddid

      AND tdspras    = iv_tdspras.

" Insert one row per plain text line

LOOP AT lt_lines INTO DATA(ls_line).

      ADD 1 TO lv_counter.

      CLEAR ls_persist.

      ls_persist-ebeln      = iv_ebeln.

      ls_persist-ebelp      = iv_ebelp.

      ls_persist-tdobject    = iv_tdoobject.

      ls_persist-tddid      = iv_tddid.

      ls_persist-tdspras     = iv_tdspras.

      ls_persist-line_counter = lv_counter.

      ls_persist-tdformat    = ls_line-tdformat.

      ls_persist-tdline      = ls_line-tdline.

      ls_persist-migrated_at = sy-datum.

      ls_persist-migrated_by = sy-uname.

      ls_persist-source      = 'D'.

      INSERT zpo_longtext FROM ls_persist.

ENDLOOP.

ENDMETHOD.

METHOD read_from_persistence.

      SELECT * FROM zpo_longtext

      INTO TABLE @rt_lines

      WHERE ebeln = @iv_ebeln

      ORDER BY tdoobject, tddid, tdspras, line_counter.

ENDMETHOD.

```

ENDCLASS.

- Step 2 complete when class is Active (blue) in SE24 with no syntax errors.

What This BAdI Does

A BAdI (Business Add-In) is a hook that SAP fires at a specific point in standard processing. ME_PROCESS_PO_CUST fires after a PO is saved in ME21N or ME22N — AFTER SAVE_TEXT has already written the text to STXH/STXL. At that point, the text exists in STXH and can be read by READ_TEXT. This BAdI calls the handler class to write the same text to ZPO_LONGTEXT in plain format.

BAdI Registration Details

Field	Value
BAdI Name	ME_PROCESS_PO_CUST
Enhancement Spot	ME_PURCHORD
Implementation Class	ZCL_PO_TEXT_BADI_IMPL
Interface	IF_EX_ME_PROCESS_PO_CUST
Methods to implement	PROCESS_HEADER (header texts), PROCESS_ITEM (item texts)

BAdI Implementation — Full ABAP Code

```

CLASS zcl_po_text_badi_impl DEFINITION PUBLIC FINAL CREATE PUBLIC.

    PUBLIC SECTION.

        INTERFACES if_ex_me_process_po_cust.

ENDCLASS.

CLASS zcl_po_text_badi_impl IMPLEMENTATION.

    " PROCESS_HEADER – fires when PO header is saved

    " Writes header texts (EKKO) to ZPO_LONGTEXT

    METHOD if_ex_me_process_po_cust~process_header.

        DATA(lv_ebeln) = im_header->get_data( )-ebeln.

        IF lv_ebeln IS INITIAL. RETURN. ENDIF.

        " Write Header text (TDID confirmed from STXH – check your system)

        zcl_po_longtext_handler=>write_to_persistence(

            iv_ebeln    = lv_ebeln

            iv_ebelp    = '000000'

```

```

        iv_tdoobject = 'EKK0'

        iv_tdid      = 'F01'    " Replace with actual TDID from STXH

        iv_tdspras   = sy-langu ).

" Write Header Delivery text if needed

zcl_po_longtext_handler=>write_to_persistence(

    iv_ebeln        = lv_ebeln

    iv_ebelp        = '00000'

    iv_tdoobject    = 'EKK0'

    iv_tdid         = 'F02'

    iv_tdspras      = sy-langu ).

ENDMETHOD.

" PROCESS_ITEM – fires when PO item is saved

" Writes item texts (EKPO) to ZPO_LONGTEXT

METHOD if_ex_me_process_po_cust~process_item.

    DATA(lv_ebeln) = im_item->get_data( )-ebeln.

    DATA(lv_ebelp) = im_item->get_data( )-ebelp.

    IF lv_ebeln IS INITIAL. RETURN. ENDIF.

" Write Item text (TDID confirmed from STXH – check your system)

zcl_po_longtext_handler=>write_to_persistence(

    iv_ebeln        = lv_ebeln

    iv_ebelp        = lv_ebelp

    iv_tdoobject    = 'EKPO'

    iv_tdid         = 'F01'    " Replace with actual TDID from STXH

    iv_tdspras      = sy-langu ).

ENDMETHOD.

ENDCLASS.

```

How to Create the BAdI in SE19

Step	Action
1	SE19 → Create Implementation → enter BAdI Name: ME_PROCESS_PO_CUST → Enter
2	Implementation Name: ZCL_PO_TEXT_BADI_IMPL → Create
3	Interface tab → IF_EX_ME_PROCESS_PO_CUST → double-click PROCESS_HEADER → paste code
4	Double-click PROCESS_ITEM → paste code
5	Save → Activate (Ctrl+F3)
6	SE19 → verify implementation shows as Active (green dot, not grey)

■ Step 3 complete when SE19 shows ZCL_PO_TEXT_BADI_IMPL as Active.

4

Test End-to-End — ME21N + SE16N Verify

Create PO with text → save → verify ZPO_LONGTEXT populated

Test Scenario — New PO with Header and Item Text

Step	Action	Detail
1	Create PO in ME21N	ME21N → enter Vendor, Purchase Org, Company Code, Document Type NB
2	Add Header text	Header → Texts → Text Overview → type in fields next to 'Header text' Line 1: Test header note line 1 Line 2: Test header note line 2
3	Add PO line item	Items section → Material, Quantity, Delivery Date, Plant
4	Add Item text	Select Item 10 → Item → Texts → Text Overview Type in fields next to 'Item text' Line 1: Test item note line 1 Line 2: Test item note line 2
5	Save PO	Ctrl+S → status bar must show: Standard PO xxxxxxxxxx saved Note the new PO number
6	Verify STXH	SE16 → STXH → TDOBJECT=EKKO, TDNAME=, Language=D → 1 row ✓ SE16 → STXH → TDOBJECT=EKPO, TDNAME=00010, Language=D → 1 row ✓
7	Verify ZPO_LONGTEXT	SE16N → ZPO_LONGTEXT → EBELN= → Execute Expect 4 rows: 2 for EKKO/F01 + 2 for EKPO/F01 TDLINE must contain plain readable text SOURCE must = D (Dual-write by BAdI)

Expected ZPO_LONGTEXT Result

SE16N → ZPO_LONGTEXT → filter EBELN = <new PO number>

EBELN	EBELP	TDOBJECT	TDID	TDSPRAS	LINE_COUNTER	TDLINE	SOURCE
<PO#>	00000	EKKO	F01	D	00001	Test header note line 1	D
<PO#>	00000	EKKO	F01	D	00002	Test header note line 2	D
<PO#>	00010	EKPO	F01	D	00001	Test item note line 1	D
<PO#>	00010	EKPO	F01	D	00002	Test item note line 2	D

If ZPO_LONGTEXT Has No Rows — Debug Checklist

Check	How	Fix
BAdI is Active	SE19 → ME_PROCESS_PO_CUST → ZCL_PO_TEXT_BADI_IMPL → status	Must show Active (green). If grey → activate.
Handler class is Active	SE24 → ZCL_PO_LONGTEXT_HANDLER → status bar	Must show Active. If inactive → Ctrl+F3 to activate.
ZPO_LONGTEXT exists	SE11 → ZPO_LONGTEXT → Display	Must exist and be Active. If not → complete Step 1.
TDID is correct	Check STXH TDID column for your PO	Update TDID in PROCESS_HEADER and PROCESS_ITEM to match.

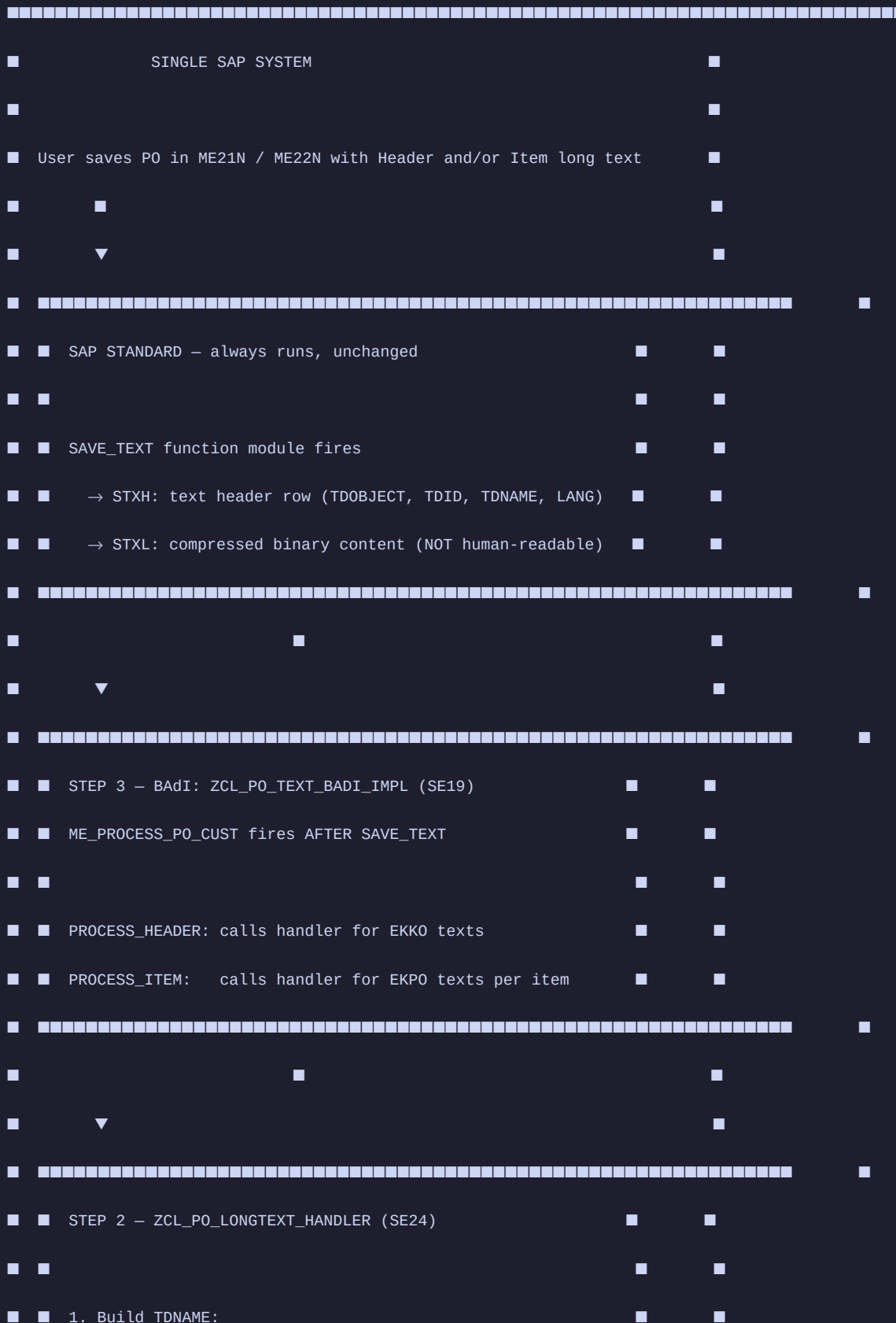
TDNAME for EKPO correct	STXH TDNAME for EKPO = 450007774400010 (no space)	Confirm CONCATENATE in handler has no space for EKPO.
PO was saved not held	Status bar after save shows 'saved' not 'held'	If held → Edit → Release Hold → Ctrl+S again.

■ Step 4 complete when ZPO_LONGTEXT shows rows with plain TDLIN text and SOURCE=D after saving a new PO with text in ME21N.

Full Picture — How All 4 Steps Work Together



The complete dual-write flow from PO save to plain SQL access



[illegible]

Key Rules — Never Forget

Rule	Detail
Never replace STXH/STXL	ZPO_LONGTEXT runs ALONGSIDE classic framework. SAP standard continues writing to STXH/STXL unchanged.
EBELP = 00000 for header	All EKKO texts (Header text, Delivery text etc.) use EBELP = 00000 in ZPO_LONGTEXT.
TDNAME for EKPO — NO SPACE	This system: CONCATENATE iv_ebeln iv_ebelp INTO lv_tlname (e.g. 450007774400010).
Always ORDER BY LINE_COUNTER	Text lines must be read in LINE_COUNTER order to reconstruct the correct text sequence.
DELETE + INSERT not UPSERT	Always delete old rows before inserting new ones — ensures deleted lines are removed too.
BAdI fires AFTER SAVE_TEXT	Text must already be in STXH before READ_TEXT is called in the BAdI — this is guaranteed.
Confirm TDID from STXH	Always verify the actual TDID from the STXH row before coding it into the handler.